

CONTENTS

Data Model — Entity Relationship & Schema Reference	
1. Overview	
2. Entity-Relationship diagram	
How the tables connect	
3. The ltree path scheme	
4. Partitioning, RLS, runtime columns & full-text — cross-cutting features	
4.1 HASH partitioning by client_id	
4.2 Row-Level Security — tenant isolation	
4.3 Runtime-added pgvector embedding columns + HNSW	
4.4 Generated full-text columns	
4.5 Soft delete	
5. Data dictionary	
5.1 di_documents — source files (002_core_tables.sql)	
5.2 doc_version — immutable version chain (002_core_tables.sql)	
5.3 di_entity — entities (002_core_tables.sql)	
5.4 client_merged_fact — client-level merged view (002_core_tables.sql)	
5.5 di_decision_trace — gate audit (002_core_tables.sql)	
5.6 knode — knowledge nodes returned to consumers (003_knode_arep.sql)	
5.7 arep — accessibility representations (003_knode_arep.sql)	
6. Relationship integrity model — why so few hard FKs	
7. Version lifecycle (how rows are produced)	

Data Model — Entity Relationship & Schema Reference

Status: Living document · **Last updated:** 2026-06-24

This document describes the Postgres 16 + pgvector + ltree schema that backs the **document_intelligence** platform: one versioned, per-client knowledge tree per KYC document, consolidated into a client-level merged-fact view. It is grounded in the committed migrations (`di/migrations/`), the runtime DDL in `di/db.py`, the domain models in `di/models.py`, and the repository SQL in `di/store.py`.

Related docs

- Design spec: [../.. / specs / 2026-06-24-document-intelligence-design.md](https://github.com/finlabs/document-intelligence/blob/main/specs/2026-06-24-document-intelligence-design.md) (§4 Data model)
 - Requirements & decisions (D1–D13): [../.. / .. / reports / requirements-and-interpretation.md](https://github.com/finlabs/document-intelligence/blob/main/reports/requirements-and-interpretation.md)
 - Migrations: [../.. / .. / di / migrations /](https://github.com/finlabs/document-intelligence/blob/main/di/migrations/)
-

1. Overview

The schema has **seven tables** living in a single configurable schema (`__SCHEMA__` , rewritten at runtime by `di/db.py:run_migrations()`):

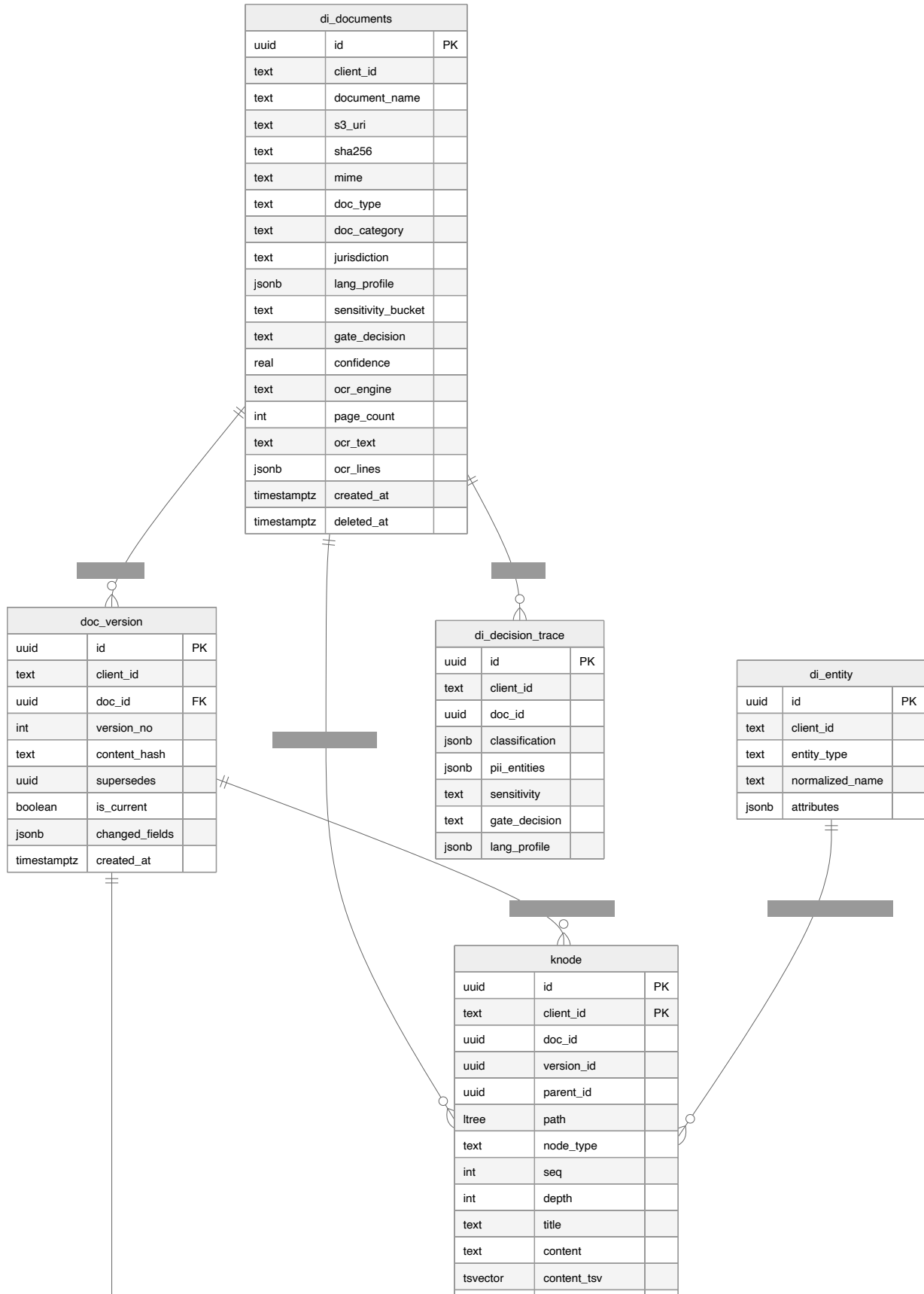
Table	Role	Volume	Partitioned
<code>di_documents</code>	One row per ingested source file	Low	No
<code>doc_version</code>	Immutable version chain per document	Low	No
<code>di_entity</code>	People / orgs / addresses within a client	Low	No
<code>client_merged_fact</code>	Client-level consolidated knowledge view	Low	No
<code>di_decision_trace</code>	Per-document PII-gate audit trail	Low	No
<code>knode</code>	Knowledge nodes returned to consumers	High	HASH by <code>client_id</code>
<code>arep</code>	Accessibility representations (searched)	High	HASH by <code>client_id</code>

The two high-volume tables — `knode` and `arep` — implement the platform’s core “index-many / return-parent” knowledge subtree (§5). They are HASH-partitioned by `client_id`. The five low-volume tables are non-partitioned but carry `client_id` and the same Row-Level Security isolation.

Every table carries a `client_id text` column and is isolated by RLS (§6). The primary tenant key arrives with the document — there is no entity-resolution step to discover it (a v1 non-goal).

2. Entity-Relationship diagram

The diagram shows key columns and the logical relationships between the tables. Note that most of these relationships are **application-enforced** rather than declared as database foreign keys — see §7 for why.



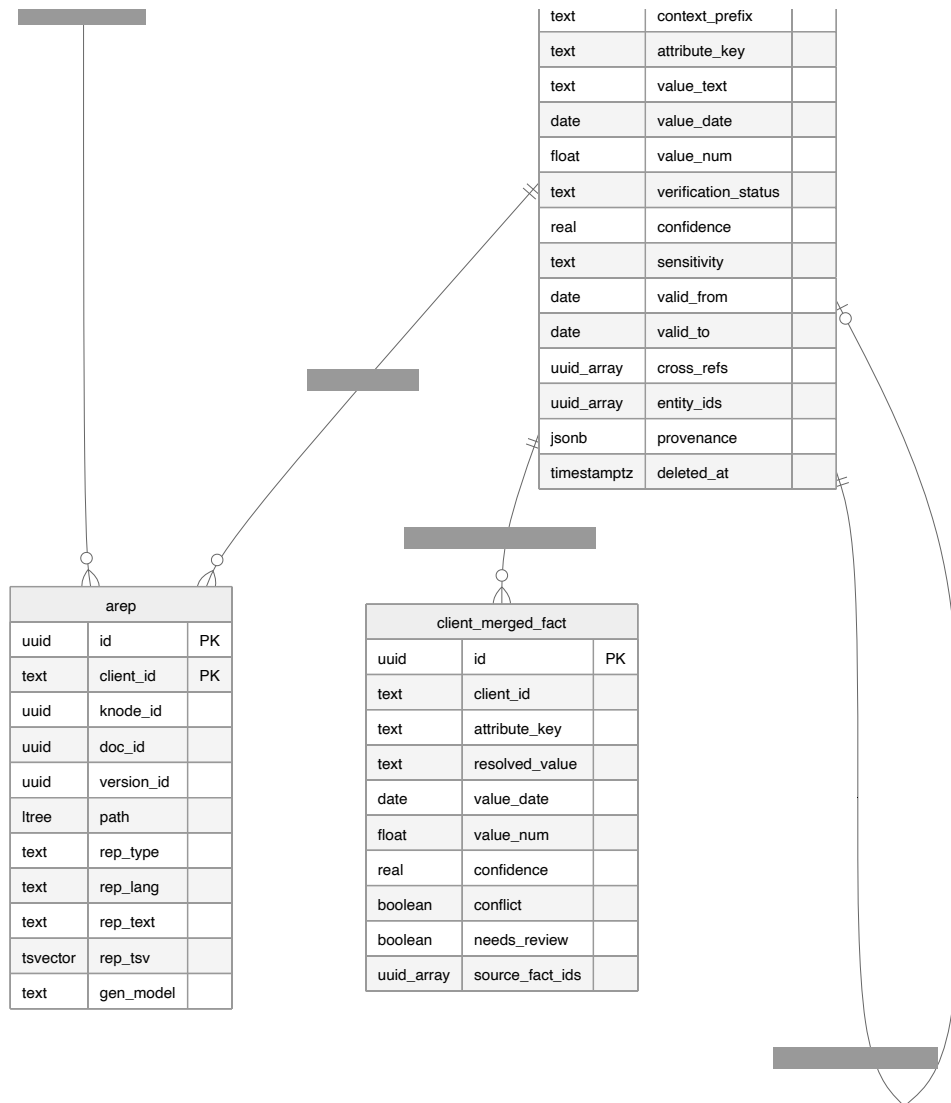


Diagram notes: Mermaid `erDiagram` has no native array type, so `uuid[]` columns are shown as `uuid_array` and double precision as `float`. `knode` and `arep` have a composite primary key (`client_id, id`) — both columns are marked `PK`. Relationship lines reflect logical joins, not declared FK constraints (only `doc_version.doc_id` is a real FK).

HOW THE TABLES CONNECT

- **di_documents** → **doc_version** (one-to-many): each logical document accumulates an immutable chain of versions. This is the **only hard foreign key** in the schema (`doc_version.doc_id REFERENCES di_documents(id) ON DELETE CASCADE`).

- **doc_version / di_documents** → **knode** and **arep**: every knowledge node and representation carries the tuple `(client_id, doc_id, version_id)`. A subtree is the set of **knode** rows sharing one `(client_id, doc_id, version_id)`. These joins are application-enforced, indexed by `knode_client_doc_ver / arep_client_knode`.
 - **knode** → **arep** (one-to-many): each **arep** row points back to its parent node via `knode_id` and copies that node's `path`, `doc_id`, and `version_id`. Reps are what searches hit; results are mapped back to the parent **knode** (the “index-many / return-parent” pattern in `di/store.py:hybrid_search`).
 - **knode** → **knode** (self-reference): `parent_id` links each node to its parent in the tree (document → section → chunk/table/figure → fact). The `ltree path` column encodes the same hierarchy redundantly for fast subtree queries.
 - **di_entity** → **knode**: a node's `entity_ids uuid[]` references the people/orgs/addresses it mentions. Many-to-many, array-valued, application-enforced.
 - **knode (fact nodes)** → **client_merged_fact**: the merge step groups **fact** nodes by `attribute_key` and writes one consolidated row per `(client_id, attribute_key)`. The contributing fact-node ids are retained in `source_fact_ids uuid[]`, giving a provenance fan-out from each merged fact back to every source fact (`di/subtree/merge.py`).
 - **di_documents** → **di_decision_trace**: one gate-decision audit row per ingested document (`doc_id` is nullable so a trace can be recorded even before the document row is committed).
-

3. The ltree path scheme

Every `knode` (and the `arep` rows that mirror it) carries an `ltree` `path` that encodes the tree position as a dot-separated label chain. The scheme is:

```
client_<id>.doctype_<type>.v<n>.<section>.<chunk-or-fact>
```

- The **root document node** path is built by `di/pipeline.py:_base_path()`: `client_<client_id>.doctype_<doc_type>.v<version_no>` (e.g. `client_42.doctype_us_passport.v1`). An unknown doc type falls back to `doctype_unknown`.
- **Child labels** are appended by `di/subtree/build.py`: sections are `s0`, `s1`, ... (one per OCR page, or a single `body` / `facts` section); chunks under a section are `c0`, `c1`, ...; facts under the synthetic `facts` section are `f0`, `f1`,
- A full chunk path therefore looks like `client_42.doctype_us_passport.v1.s0.c2` and a fact like `client_42.doctype_us_passport.v1.s1.f0`.

Label sanitisation (`build.sanitize_label`): every label is lowercased, any run of characters outside `[A-Za-z0-9_]` is collapsed to a single `_`, repeated underscores are merged, and leading/trailing underscores are stripped. An empty or all-unsafe label falls back to `x`, so the result is always a valid, non-empty ltree label. `depth` is stored alongside the path and equals the ltree `nlevel` (count of dot-separated labels).

Why ltree. The path gives three things cheaply: (1) ancestor/descendant containment queries via the GiST index (`path <@ '<prefix>':::ltree`, used for scoped search and `fetch_subtree`); (2) a stable, human-readable address for any node; and (3) the “logical” property of the knowledge subtree (`di/store.py` scopes searches by `scope_path`). The `parent_id` self-reference and the `path` are deliberately redundant — `parent_id` is the source of truth for tree linkage, `path` optimises range/containment reads.

4. Partitioning, RLS, runtime columns & full-text — cross-cutting features

4.1 HASH PARTITIONING BY CLIENT_ID

`knode` and `arep` are declared `PARTITION BY HASH (client_id)`. The partitions themselves are created **programmatically** at startup by

`di/db.py:_create_hash_partitions()`, which creates `<table>_p0 ... <table>_p{N-1}` with `FOR VALUES WITH (MODULUS N, REMAINDER i)`, where `N` is `settings.pg_hash_partitions` (the design spec suggests a fixed count such as 64). Indexes declared on the partitioned parent propagate to all current and future partitions.

This keeps each tenant's high-volume rows physically grouped, bounds the size of any one B-tree / GiST / GIN / HNSW index, and pairs naturally with the `client_id`-leading composite primary key `(client_id, id)` — the partition key must be part of the primary key.

4.2 ROW-LEVEL SECURITY — TENANT ISOLATION

`di/migrations/004_rls.sql` runs a `DO` block over **all seven tables** that, for each, enables and `FORCE`s Row-Level Security and creates a `tenant_isolation` policy:

```
USING      (client_id = current_setting('app.current_client_id', true))
WITH CHECK (client_id = current_setting('app.current_client_id', true))
```

The per-connection GUC `app.current_client_id` is bound on every pool checkout by `di/db.py:acquire(client_id=...)` and reset to empty on release. `FORCE` means even the table owner is filtered. Caveat (documented in the migration): superusers and `BYPASSRLS` roles bypass RLS even with `FORCE`; in production the app connects as a non-superuser role, while local dev may run as a superuser, in which case isolation relies on the application always passing `client_id`.

4.3 RUNTIME-ADDED PGVECTOR EMBEDDING COLUMNS + HNSW

The base extensions migration (`001_extensions.sql`) creates only `ltree` and `pgcrypto`. The `vector` type, the embedding columns, and the HNSW indexes are **intentionally not** in the migrations — they are added at runtime by `di/db.py:_ensure_vector_columns()` once the live embedding dimension is known (typically discovered from the retrieval service's `/api/models`):

- `ALTER TABLE knode ADD COLUMN content_embedding vector(D)`
- `ALTER TABLE arep ADD COLUMN rep_embedding vector(D)`
- one HNSW index **per partition** (`<table>_p{i}<col>_hnsw`) using `vector_cosine_ops`, because pgvector cannot build an HNSW index directly on a partitioned parent.

When pgvector is absent (e.g. local dev without the extension), these steps are skipped cleanly and the system degrades to full-text-only search. The repository (`di/store.py`) checks `pgvector_available()` at insert and query time and only includes the embedding columns/legs when the extension is present.

4.4 GENERATED FULL-TEXT COLUMNS

Both high-volume tables carry a `GENERATED ALWAYS AS ... STORED` `tsvector` column for full-text search, each backed by a GIN index:

- `knode.content_tsv = to_tsvector('simple', coalesce(title, '') || ' ' || coalesce(content, ''))`
- `arep.rep_tsv = to_tsvector('simple', coalesce(rep_text, ''))`

The `simple` configuration is used deliberately (no stemming / stop-word removal) so the same index serves English and Spanish content. Lexical search uses `websearch_to_tsquery('simple', ...)` ranked by `ts_rank`, fused with the vector legs via Reciprocal Rank Fusion in `hybrid_search`.

4.5 SOFT DELETE

`di_documents` and `knode` carry a nullable `deleted_at timestampz`. Deletion is logical: rows are stamped, not removed, and every read filters `deleted_at IS NULL` (the `di_documents_client*` and search/subtree queries do this). Partial indexes (`WHERE deleted_at IS NULL`) keep the live working set lean. Soft delete preserves provenance and the audit trail for compliance (design spec §9).

5. Data dictionary

Types and notes are read directly from the migrations. `__SCHEMA__` is the configured schema.

5.1 `DI_DOCUMENTS` — SOURCE FILES (`002_CORE_TABLES.SQL`)

One row per source file ingested for a client. UPSERTed by `(client_id, document_name)`.

Column	Type	Notes
<code>id</code>	<code>uuid</code> PK	<code>DEFAULT</code> <code>gen_random_uuid()</code>
<code>client_id</code>	<code>text</code> NOT NULL	Tenant key; RLS filter
<code>document_name</code>	<code>text</code> NOT NULL	Unique within client (<code>UNIQUE</code> (<code>client_id</code> , <code>document_name</code>))
<code>s3_uri</code>	<code>text</code>	Source-file location (S3 / MinIO)
<code>sha256</code>	<code>text</code>	File hash; basis for re-upload dedup
<code>mime</code>	<code>text</code>	Detected MIME type
<code>doc_type</code>	<code>text</code>	Classifier output, e.g. <code>US_PASSPORT</code>
<code>doc_category</code>	<code>text</code>	identity / address / income / tax / corporate / bank_form
<code>subject</code>	<code>text</code>	Free-text subject of the document
<code>jurisdiction</code>	<code>text</code>	US / CA / MX
<code>lang_profile</code>	<code>jsonb</code> NOT NULL <code>DEFAULT '{}'</code>	Dominant + per-span languages
<code>sensitivity_bucket</code>	<code>text</code> NOT NULL <code>DEFAULT 'LOW'</code>	LOW / MEDIUM / HIGH / CRITICAL
<code>gate_decision</code>	<code>text</code>	<code>SEND_TO_LLM</code> / <code>REDACT_THEN_SEND</code> / <code>DETERMINISTIC_ONLY</code>
<code>confidence</code>	<code>real</code> NOT NULL <code>DEFAULT 0</code>	Classification confidence
<code>ocr_engine</code>	<code>text</code>	Engine used (Azure Read / fallback)

Column	Type	Notes
<code>page_count</code>	<code>int</code>	Pages processed
<code>ocr_text</code>	<code>text</code>	Full concatenated OCR text
<code>ocr_lines</code>	<code>jsonb</code> NOT NULL DEFAULT <code>'[]'</code>	Per-line text + bbox + confidence (feeds deterministic extraction)
<code>classification_signals</code>	<code>jsonb</code> NOT NULL DEFAULT <code>'[]'</code>	Anchor/classifier evidence strings
<code>created_at</code>	<code>timestampz</code> NOT NULL DEFAULT <code>now()</code>	
<code>updated_at</code>	<code>timestampz</code> NOT NULL DEFAULT <code>now()</code>	Bumped on UPSERT
<code>deleted_at</code>	<code>timestampz</code>	Soft delete; <code>NULL</code> = live

Indexes: `di_documents_client (client_id) WHERE deleted_at IS NULL`,
`di_documents_client_type (client_id, doc_type) WHERE deleted_at IS NULL`.

5.2 `DOC_VERSION` — IMMUTABLE VERSION CHAIN (`002_CORE_TABLES.SQL`)

Column	Type	Notes
<code>id</code>	<code>uuid</code> PK	DEFAULT <code>gen_random_uuid()</code>
<code>client_id</code>	<code>text</code> NOT NULL	Tenant key
<code>doc_id</code>	<code>uuid</code> NOT NULL	FK → <code>di_documents(id)</code> ON DELETE CASCADE
<code>version_no</code>	<code>int</code> NOT NULL	Monotonic per document, starting at 1
<code>content_hash</code>	<code>text</code> NOT NULL	Dedup key; identical re-upload is a no-op
<code>supersedes</code>	<code>uuid</code>	Previous version's id (chain pointer)
<code>is_current</code>	<code>boolean</code> NOT NULL DEFAULT <code>true</code>	Exactly one current per doc
<code>changed_fields</code>	<code>jsonb</code> NOT NULL DEFAULT <code>'[]'</code>	Diff vs prior version
<code>created_at</code>	<code>timestampz</code> NOT NULL DEFAULT <code>now()</code>	
<code>created_by</code>	<code>text</code>	Actor that created the version

Indexes: partial unique `doc_version_one_current (client_id, doc_id) WHERE is_current` (enforces a single current version per document), `doc_version_doc (client_id, doc_id)`.

5.3 `DI_ENTITY` — ENTITIES (`002_CORE_TABLES.SQL`)

Referenced by `knode.entity_ids`. People / orgs / addresses within a client.

Column	Type	Notes
<code>id</code>	<code>uuid</code> PK	<code>DEFAULT</code> <code>gen_random_uuid()</code>
<code>client_id</code>	<code>text</code> NOT NULL	Tenant key
<code>entity_type</code>	<code>text</code> NOT NULL	person / org / address / ...
<code>normalized_name</code>	<code>text</code>	Canonicalised name
<code>attributes</code>	<code>jsonb</code> NOT NULL <code>DEFAULT</code> <code>'{}'</code>	Flexible per-type payload
<code>created_at</code>	<code>timestampz</code> NOT NULL <code>DEFAULT now()</code>	

Index: `di_entity_client (client_id, entity_type)`.

5.4 `CLIENT_MERGED_FACT` — CLIENT-LEVEL MERGED VIEW (`002_CORE_TABLES.SQL`)

Intra-client consolidation of `fact` knodes; confidence-weighted (highest-confidence source wins, disagreements flagged). UPSERTed by `(client_id, attribute_key)`.
Rebuildable from `knode` facts.

Column	Type	Notes
<code>id</code>	<code>uuid</code> PK	<code>DEFAULT gen_random_uuid()</code>
<code>client_id</code>	<code>text</code> NOT NULL	Tenant key
<code>attribute_key</code>	<code>text</code> NOT NULL	Canonical key, e.g. <code>identity.date_of_birth</code> (<code>di/ontology.py</code>)
<code>resolved_value</code>	<code>text</code>	Winning value (string form)
<code>value_date</code>	<code>date</code>	Winning date value
<code>value_num</code>	<code>double precision</code>	Winning numeric value
<code>confidence</code>	<code>real</code> NOT NULL <code>DEFAULT 0</code>	Winner's confidence
<code>conflict</code>	<code>boolean</code> NOT NULL <code>DEFAULT false</code>	Sources disagree on a non-empty value
<code>needs_review</code>	<code>boolean</code> NOT NULL <code>DEFAULT false</code>	Human-review flag (set on conflict)
<code>source_fact_ids</code>	<code>uuid[]</code> NOT NULL <code>DEFAULT '{}'</code>	All contributing <code>knode</code> fact ids (provenance fan-out)
<code>updated_at</code>	<code>timestampz</code> NOT NULL <code>DEFAULT</code> <code>now()</code>	Bumped on UPSERT

Constraint: `UNIQUE (client_id, attribute_key)`. Index:
`client_merged_fact_client (client_id)`.

5.5 `DI_DECISION_TRACE` — GATE AUDIT (`002_CORE_TABLES.SQL`)

One row per document gate decision, for compliance.

Column	Type	Notes
<code>id</code>	<code>uuid</code> PK	DEFAULT <code>gen_random_uuid()</code>
<code>client_id</code>	<code>text</code> NOT NULL	Tenant key
<code>doc_id</code>	<code>uuid</code>	Nullable (trace may precede the doc row)
<code>classification</code>	<code>jsonb</code> NOT NULL DEFAULT <code>'{}'</code>	Classifier output + confidence
<code>pii_entities</code>	<code>jsonb</code> NOT NULL DEFAULT <code>'[]'</code>	Detected PII entities + scores
<code>sensitivity</code>	<code>text</code>	LOW ... CRITICAL
<code>gate_decision</code>	<code>text</code>	SEND_TO_LLM / REDACT_THEN_SEND / DETERMINISTIC_ONLY
<code>lang_profile</code>	<code>jsonb</code> NOT NULL DEFAULT <code>'{}'</code>	Language detection result
<code>created_at</code>	<code>timestampz</code> NOT NULL DEFAULT <code>now()</code>	

Index: `di_decision_trace_client (client_id, created_at DESC)`.

5.6 `KNODE` — KNOWLEDGE NODES RETURNED TO CONSUMERS (`003_KNODE_AREP.SQL`)

HASH-partitioned by `client_id`. Composite primary key `(client_id, id)`. These are the canonical, returnable logical & content nodes of the knowledge subtree.

Column	Type	Notes
<code>id</code>	<code>uuid</code> NOT NULL	DEFAULT <code>gen_random_uuid()</code> ; part of composite PK
<code>client_id</code>	<code>text</code> NOT NULL	Tenant key; partition key; part of composite PK
<code>doc_id</code>	<code>uuid</code> NOT NULL	Owning document
<code>version_id</code>	<code>uuid</code> NOT NULL	Owning version
<code>parent_id</code>	<code>uuid</code>	Self-reference to parent node (<code>NULL</code> for the document root)
<code>path</code>	<code>ltree</code> NOT NULL	Hierarchical path (§3)
<code>node_type</code>	<code>text</code> NOT NULL	document / section / chunk / table / figure / fact / summary
<code>seq</code>	<code>int</code> NOT NULL DEFAULT <code>0</code>	Sibling order
<code>depth</code>	<code>int</code> NOT NULL DEFAULT <code>0</code>	Equals <code>nlevel(path)</code>
<code>title</code>	<code>text</code>	Node title / fact attribute label
<code>content</code>	<code>text</code>	Node text content
<code>content_tsv</code>	<code>tsvector</code> GENERATED STORED	FTS over `title`
<code>context_prefix</code>	<code>text</code>	Contextual lead-in (the “contextual” property)
<code>attribute_key</code>	<code>text</code>	Canonical key for <code>fact</code> nodes
<code>value_text</code>	<code>text</code>	Fact value (string)
<code>value_date</code>	<code>date</code>	Fact value (date)
<code>value_num</code>	<code>double precision</code>	Fact value (numeric)

Column	Type	Notes
<code>verification_status</code>	<code>text</code> NOT NULL DEFAULT <code>'unverified'</code>	checksum_verified / gov_verified / llm_unverified / unverified
<code>confidence</code>	<code>real</code> NOT NULL DEFAULT <code>0</code>	Extraction confidence
<code>sensitivity</code>	<code>text</code> NOT NULL DEFAULT <code>'LOW'</code>	Per-node PII level (drives masking)
<code>valid_from</code>	<code>date</code>	Real-world fact validity start (time- travel)
<code>valid_to</code>	<code>date</code>	Real-world fact validity end (time- travel)
<code>cross_refs</code>	<code>uuid[]</code> NOT NULL DEFAULT <code>'{}'</code>	DAG cross-references to other nodes
<code>entity_ids</code>	<code>uuid[]</code> NOT NULL DEFAULT <code>'{}'</code>	References <code>di_entity</code> rows
<code>provenance</code>	<code>jsonb</code> NOT NULL DEFAULT <code>'{}'</code>	page / bbox / char-span / extractor / model
<code>token_count</code>	<code>int</code>	Estimated tokens
<code>created_at</code>	<code>timestampz</code> NOT NULL DEFAULT <code>now()</code>	
<code>deleted_at</code>	<code>timestampz</code>	Soft delete
<code>content_embedding</code>	<code>vector(D)</code>	Added at runtime by <code>di/db.py</code> when pgvector present

Indexes: `knode_path_gist` GiST(`path`), `knode_tsv_gin` GIN(`content_tsv`),
`knode_client_path` (`client_id`, `path`), `knode_client_doc_ver` (`client_id`,
`doc_id`, `version_id`), `knode_client_type` (`client_id`, `node_type`), partial
`knode_attr` (`client_id`, `attribute_key`) WHERE `node_type` = 'fact' , plus per-
partition `..._content_embedding_hnsw` (runtime, when pgvector present).

5.7 AREP — ACCESSIBILITY REPRESENTATIONS

(003_KNODE_AREP.SQL)

HASH-partitioned by `client_id`. Composite primary key `(client_id, id)`. The “searched” side of the index-many / return-parent pattern: each rep maps back to its parent `knode` via `knode_id`.

Column	Type	Notes
<code>id</code>	<code>uuid</code> NOT NULL	DEFAULT <code>gen_random_uuid()</code> ; part of composite PK
<code>knode_id</code>	<code>uuid</code> NOT NULL	Parent node this rep represents
<code>client_id</code>	<code>text</code> NOT NULL	Tenant key; partition key; part of composite PK
<code>doc_id</code>	<code>uuid</code> NOT NULL	Copied from the parent node
<code>version_id</code>	<code>uuid</code> NOT NULL	Copied from the parent node
<code>path</code>	<code>ltree</code> NOT NULL	Copied from the parent node
<code>rep_type</code>	<code>text</code> NOT NULL	hypothetical_q / proposition / summary / alt_phrasing / synonym_expansion / table_desc / figure_desc / keyword_set / translation
<code>rep_lang</code>	<code>text</code> NOT NULL DEFAULT <code>'en'</code>	Representation language (enables cross-lingual EN↔ES)
<code>rep_text</code>	<code>text</code> NOT NULL	The representation text
<code>rep_tsv</code>	<code>tsvector</code> GENERATED STORED	FTS over <code>rep_text</code> (simple)
<code>gen_model</code>	<code>text</code>	Model that produced the rep
<code>created_at</code>	<code>timestampz</code> NOT NULL DEFAULT <code>now()</code>	
<code>rep_embedding</code>	<code>vector(D)</code>	Added at runtime by <code>di/db.py</code> when <code>pgvector</code> present

Indexes: `arep_path_gist` GiST(`path`), `arep_tsv_gin` GIN(`rep_tsv`), `arep_client_knode` (`client_id`, `knode_id`), `arep_client_type` (`client_id`, `rep_type`), plus per-partition `..._rep_embedding_hnsw` (runtime, when `pgvector` present).

6. Relationship integrity model — why so few hard FKs

Only one declared foreign key exists in the schema: `doc_version.doc_id → di_documents(id)`. Every other relationship (`knode.parent_id`, `knode/arep → version/document`, `knode.entity_ids[]`, `arep.knode_id`, `client_merged_fact.source_fact_ids[]`) is **application-enforced**. This is deliberate:

- **Partitioning constraints.** `knode` and `arep` are HASH-partitioned by `client_id`. A foreign key referencing a partitioned table must include the full partition key, and Postgres support for FKs *referencing* partitioned tables is limited. A self-referential FK on `knode.parent_id` would also need to span partitions. The cost of these constraints on high-volume insert paths is not worth it.
- **Array-valued links.** `entity_ids[]`, `cross_refs[]`, and `source_fact_ids[]` are arrays — Postgres cannot express a foreign key over array elements. The application is responsible for populating them with valid ids.
- **RLS already scopes every join.** Because each tenant is isolated by RLS and every query is filtered to one `client_id`, dangling references cannot cross tenants and the join surface is small.
- **Ordering / decoupling during ingest.** A `di_decision_trace` may be written before the `di_documents` row exists (`doc_id` is nullable), so a hard FK there would be a liability.

The trade-off is that referential integrity for these links is the ingestion code's responsibility (`di/subtree/*`, `di/store.py`). In exchange the high-volume write paths stay fast and partition- friendly. Reads are kept consistent by always scoping to (`client_id`, ...) and, for the live view, filtering to the current version via the `is_current` predicate (`di/store.py:_current_clause`).

7. Version lifecycle (how rows are produced)

1. **Ingest** (`di/pipeline.py:ingest_document`) UPSErTs a `di_documents` row, then plans a version (`di/subtree/versioning.py`): identical `content_hash` $\Rightarrow$ no-op; otherwise `version_no = previous + 1`, the prior current version's `is_current` flips to `false`, and a new `doc_version` row is inserted (`is_current = true`).
2. The subtree (`di/subtree/build.py`) is assembled with the version's `base_path` and persisted as `knode` rows; representations become `arep` rows.
3. Fact nodes are merged (`di/subtree/merge.py`) into `client_merged_fact`, retaining all source fact ids.
4. Soft-deleted documents/nodes (`deleted_at`) are excluded from all reads but retained for audit.

This keeps the “as-of” and “what-changed” capabilities (design spec §6) backed purely by the version chain plus the per-fact `valid_from` / `valid_to` columns.